

Pre Lab

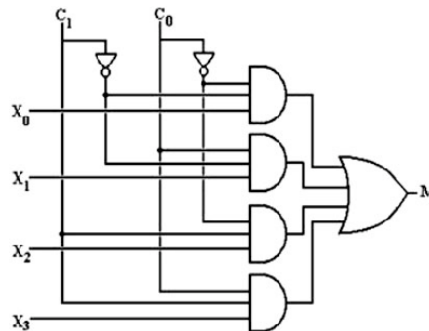
1. Explain the fundamental operation of a multiplexer and how it functions as a data selector in digital circuits.

The multiplexer has ports for input and ports for a selector input. The selector inputs can be described as a binary selector as each combination represents an input port. By changing the selector combination input you can select which data is passed to the output.

2. Derive the Boolean expression for a 4:1 multiplexer from its truth table and draw its gate-level Implementation.

S1	S0	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

$$Y = (\sim S1) (\sim S0) D0 + (\sim S1) S0 D1 + S1 (\sim S0) D2 + S1 S0 D3$$



3. What are the key advantages of using `always_comb` over `always @(*)` in SystemVerilog for combinational logic design?

`always_comb` in SystemVerilog is preferred over `always @(*)` because it provides stricter and safer semantics for combinational logic. It automatically includes all signals in its sensitivity list, prevents unintended latches by ensuring all outputs are assigned in every path, disallows timing or event controls, and clearly expresses the designer's intent. These checks reduce simulation–synthesis mismatches and make the code more reliable and easier to debug.

4. How does the number of selection lines relate to the number of inputs in a multiplexer? Provide the formula and calculate the selection lines needed for a 16:1 MUX.

$$- 2^n = \text{number of inputs} \quad n = \log_2(\text{number of inputs})$$
$$n = \log_2(16) = 4$$

So a 16:1 multiplexer needs **4 selection lines** (bits) to select one of the 16 inputs.

PostLab

1. Compare the advantages and disadvantages of the three Verilog modeling styles (Dataflow, Behavioral, Structural) for multiplexer design. When would you prefer one over the others?

In Verilog multiplexer design, dataflow modeling expresses the logic using Boolean equations or assign statements, making it concise and efficient for simple combinational circuits but harder to manage in large designs. Behavioral modeling uses always_comb or case statements, offering readability and flexibility for complex logic, though it risks creating unintended latches if not written carefully. Structural modeling connects logic gates or submodules to show the actual hardware structure, providing clear hierarchy and accuracy but making the code lengthy and difficult to scale for larger multiplexers. For me the I prefer using behavioral design as it is the most understandable for me

2. Explain the difference between blocking and non-blocking assignments in Verilog. Why must we use blocking assignments for the combinational logic in this lab?

In Verilog, blocking assignments (=) execute sequentially within an always block, meaning each statement must finish before the next begins, while non-blocking assignments (<=) execute concurrently, allowing all updates to happen at the end of the simulation time step. In combinational logic, blocking assignments are used because they ensure that signals are updated immediately and reflect proper logic flow, preventing unintended timing or latch behavior that could occur with non-blocking assignments.

3. What are inferred latches, how are they created, and how can they be avoided in combinational logic design?

An inferred latch occurs when a signal in a combinational always block is not assigned in all possible conditions, causing it to retain its previous value like a storage element. For example, missing an else branch in an if statement can infer a latch. To avoid them, ensure all outputs are assigned a value on every execution path—typically by using always_comb, complete if-else or case statements, or default assignments at the start of the block.

4. Describe how you would implement an 8:1 multiplexer using only 2:1 multiplexers. How many 2:1 MUXes would you need?

First stage: 4 MUX select between 8 inputs (I0–I7) → 4 outputs.

Second stage: 2 MUX select between those 4 outputs → 2 outputs.

Final stage: 1 MUX selects between those 2 outputs → final output.

Total 2:1 MUXes needed = $4 + 2 + 1 = 7$.

5. Look at the waveforms from your structural 4:1 MUX (built from gates) and your behavioral 4:1 MUX (using case). Do the outputs change at exactly the same time in simulation? Why might there be a difference?

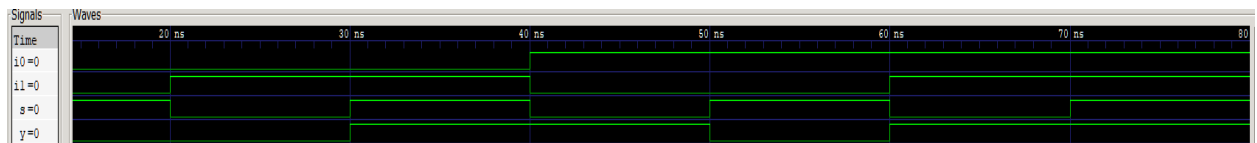
The structural 4:1 MUX (built from gates) and the behavioral 4:1 MUX (using case) may not have outputs that change at exactly the same time in simulation because gate-level models include propagation delays, while behavioral models are often simulated with zero delay. This difference causes the structural MUX to show slightly delayed output transitions compared to the ideal, instantaneous response of the behavioral MUX.

CODE IN GITHUB

<https://github.com/heapdats/COE180.1>

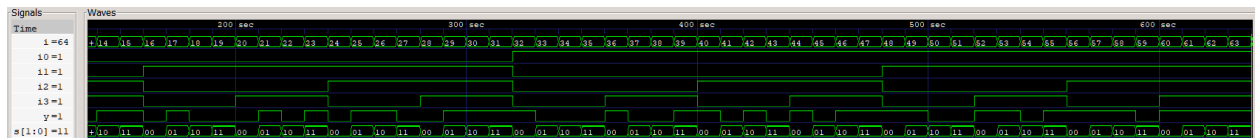
GTK Wave

2:1 Multiplexer



At 30ns $s=1$, $i_1=1$ and $y=1$, At 40ns $s=0$, $i_0=1$ and $y=1$

4:1 Multiplexer



0–150 ns: $s = 00 \rightarrow$ output y follows i_0 .

150–300 ns: $s = 01 \rightarrow$ y follows i_1 .

300–450 ns: $s = 10 \rightarrow$ y follows i_2 .

450–600 ns: $s = 11 \rightarrow$ y follows i_3 .

8:1 Multiplexer

